Improving Text Embeddings for Smaller Language Models Using Contrastive Fine-tuning

Trapoom Ukarapol, Zhicheng Lee, Amy Xin
Department of Computer Science and Technology, Tsinghua University
{ukarapolt10, lizhiche23, xin-x23}@mails.tsinghua.edu.cn

arXiv:2408.00690v2 [cs.CL] 2 Aug 2024

Abstract

While Large Language Models show remarkable performance in natural language understanding, their resource-intensive nature makes them less accessible. In contrast, smaller language models such as MiniCPM offer more sustainable scalability, but often underperform without specialized optimization. In this paper, we explore the enhancement of smaller language models through the improvement of their text embeddings. We select three language models, MiniCPM, Phi-2, and Gemma, to conduct contrastive fine-tuning on the NLI dataset. Our results demonstrate that this fine-tuning method enhances the quality of text embeddings for all three models across various benchmarks, with MiniCPM showing the most significant improvements of an average 56.33% performance gain. The contrastive fine-tuning code is publicly available at https://github.com/trapoom555/Language-Model-STS-CFT.

1 Introduction

Text embeddings are vector representations of text data that encodes semantic information, enabling machines to understand and process natural language. These embeddings are crucial for a variety of tasks including document classification, semantic similarity matching, and information retrieval. With the recent advancements of Large Language Models (LLMs), LLMs such as GPT-4 (Achiam et al., 2023), LLaMA (Touvron et al., 2023), and Mistral (Jiang et al., 2023a) have demonstrated robust natural language understanding capabilities due to their large-scale training. Conversely, smaller models like Gemma (Team et al., 2024), Phi (Li et al., 2023), and MiniCPM (Hu et al., 2024), while less resource-intensive, often lag in performance without specific optimizations.

This project is motivated by the lack of research focused on enhancing the text embedding capabilities of relatively smaller models. Notably, MiniCPM, a smaller-scale language model, underperforms in generating effective text embeddings without further fine-tuning. Addressing this gap, we aim to conduct experiments to improve the quality of text embeddings generated by MiniCPM, thereby making it a viable option for resource-constrained applications. This project could pave the way for using small scale models on the text embedding task. Furthermore, we compare MiniCPM's performance among other small language models including Gemma and Phi-2 after fine-tuning with the same dataset.

2 Related Works

Text Embeddings Text embeddings are vector representations of text in a low-dimensional space. These embeddings are designed to encapsulate the semantic meaning of the text, facilitating various downstream tasks including information retrieval, document classification, and similarity matching. Traditional models such as SBERT (Reimers and Gurevych, 2019) and Sentence T5 (Ni et al., 2021) have aimed to provide a universal framework for encoding the semantic significance of text across different tasks and domains. Moreover, methods such as Contriever (Izacard et al., 2021), E5 (Wang et al., 2022), and SGPT (Muennighoff, 2022) have implemented a multi-stage training strategy to enhance the effectiveness of text embeddings. To assess the performance of text embeddings within the proposed model, the benchmark MTEB (Muennighoff et al., 2022) is utilized. This benchmark includes novel tasks to evaluate the text embedding quality of the models and their abilities to generalize across different domains.

Contrastive Representation Learning Contrastive Representation Learning aims to learn effective representation by pulling semantically close neighbors together and pushing apart non-neighbors (Hadsell et al., 2006). Early works

learn such representation by using contrastive loss (Chopra et al., 2005) and triplet loss (Schroff et al., 2015), where only one positive and one negative sample are involved. On top of that, multi-class N-pair loss (Sohn, 2016) and InfoNCE loss (Oord et al., 2018) include multiple positive and negative pairs in a batch to improve the semantic representation of the embeddings.

Lightweight LLMs Despite the remarkable performance of LLMs in text generation tasks, the cost of training, serving and maintaining such large scale model is huge. Therefore, several lightweight LLMs within the range of 1 to 3 billion parameters have been proposed to tackle this issue. Notable examples include the Phi series (Li et al., 2023), Gemma (Team et al., 2024) and MiniCPM (Hu et al., 2024). These lightweight language models require fewer resources, thereby reducing the barriers to entry for conducting studies with smaller-scale models.

LLM Fine-tuning In the recent year, Pre-training and Fine-tuning Paradigm has been successfully proven to be better than training from scratch. The foundational work, Bidirectional Encoder Representations from Transformers (BERT) (Devlin et al., 2019) established the pre-training and fine-tuning paradigm, where models are first pre-trained on a large corpus of text and then fine-tuned on specific downstream tasks. This approach has become a cornerstone for many subsequent models, including GPT-3 (Brown et al., 2020), which demonstrated that increasing model size and the amount of pre-training data can lead to significant improvements in performance across a wide range of tasks. Recent research has focused on making fine-tuning more efficient, particularly for large-scale models. Techniques such as Adapter modules, proposed by (Houlsby et al., 2019), involve inserting small trainable layers within the pre-trained model, allowing for task-specific adaptation without modifying the entire model's parameters. This approach significantly reduces the computational cost of fine-tuning. Another notable technique is Low-Rank Adaptation (LoRA), introduced by (Hu et al., 2021), which adapts the weights of the model using low-rank matrix factorization. This method achieves substantial parameter efficiency, enabling the fine-tuning of large models with a fraction of the computational resources typically required.

3 Methodology

Our approach aims to address the problem of Semantic Textual Similarity (STS) in the English language by employing smaller language models, thereby ensuring that our solution is both efficient and scalable. To enhance the text embedding capabilities of these smaller models, we utilize a contrastive fine-tuning approach. Contrastive fine-tuning involves training the model to distinguish between similar and dissimilar pairs of text. This method helps the model to generate more accurate and contextually relevant embeddings, which are essential for assessing semantic similarity. The details of contrastive fine-tuning are described in section 3.3.

Additionally, we adopt a parameter-efficient fine-tuning technique. This technique is designed to achieve optimal performance while minimizing the computational resources required. By using the low-rank adaptation (LoRA) (Hu et al., 2021) method, we ensure that our approach remains computationally feasible even with limited hardware resources. This is particularly important for fine-tuning the models in the scenario where computational efficiency is a key concern.

3.1 Dataset

The processed NLI dataset (Oord et al., 2018) is adopted as our training dataset. There are approximately 275k samples in the dataset. Each entry includes a premise x_i, along with its corresponding entailment x_i^+ and contradiction x_i^-, thus forming a triplet (x_i, x_i^+, x_i^-)

3.2 Language Model Choices

We conduct our experiments on language models with fewer parameters up to 2B, including Gemma (Team et al., 2024), Phi-2 (Li et al., 2023), and MiniCPM (Hu et al., 2024) to explore their capabilities on the text embedding task.

3.3 Contrastive Fine-tuning

In this study, we leverage the language understanding capabilities of language models (LMs) and enhance their text embedding quality using a contrastive fine-tuning paradigm. To achieve higher text embedding quality with limited computational resources, we adopt a parameter-efficient fine-tuning technique, LoRA (Hu et al., 2021), as our fine-tuning method. The details of how text embeddings are extracted from the Language Model

(LM) and the training objective are described in the following sections.

Embedding Vector Extraction Given a pre-trained language model and a prompt, we augment the prompt by appending the <EOS> token. Subsequently, we input this modified prompt into the language model. The embedding vector can then be acquired by extracting the vector corresponding to the final <EOS> token from the last layer.

Training Objective We utilize the standard InfoNCE objective (Oord et al., 2018) with in-batch negatives and hard negatives. The training objective can be described by the following expression.

$$\min -\log \frac{e^{\text{sim}(\mathbf{h}_i, \mathbf{h}_i^+)/\tau}}{\sum_{j=1}^{N} \left(e^{\text{sim}(\mathbf{h}_i, \mathbf{h}_j^+)/\tau} + e^{\text{sim}(\mathbf{h}_i, \mathbf{h}_j^-)/\tau} \right)} \tag{1}$$

where $\mathbf{h}_i$ denotes an embedding vector of a premise x_i, τ denotes a temperature and $\text{sim}(\mathbf{h}_i, \mathbf{h}_i^+)$ computes the cosine similarity between embedding vectors $\mathbf{h}_i$ and $\mathbf{h}_i^+$.

4 Experiments

Given a pair of sentences, the STS task aims to measure the similarity score of the embeddings embeded by the models where higher scores signify greater similarity. Specifically, we measure the distance between embeddings using cosine similarity, then calculate the correlation with ground truth similarities using Spearman correlations.

We compare the performance of MiniCPM and baseline models such as Gemma and Phi-2 on 9 benchmarks, namely STS12 (Agirre et al., 2012), STS13 (Agirre et al., 2013), STS14[1], STS15[2], STS16[3], STS17[4], STSBenchmark, BIOSSES[5] and SICK-R (Agirre et al., 2014). These datasets consist of pairs of sentences with similarity scores from 0 to 5. To ensure fairness, both MiniCPM and the baseline models are fine-tuned on the same processed NLI dataset.

STS12, STS13, STS14, STS15, STS16, STS17, STSBenchmark This set of STS benchmarks incorporate sentences derived from various sources including image captions, news headlines, and user forums. These datasets range from 1,000 to 20,000 sentences.

BIOSSES The BIOSSES benchmark contains 100 sentence pairs from the biomedical field.

SICK-R The Sentences Involving Compositional Knowledge (SICK) dataset comprises a substantial corpus of 100,000 sentence pairs, each characterized by its lexical, syntactic, and semantic richness.

4.1 Main Experiment Results

Table 2 presents the evaluation results of three models: Gemma, Phi-2, and MiniCPM across all 9 benchmarks, using Spearman correlations of cosine similarities post-supervised fine-tuning (SFT). The results demonstrate that MiniCPM consistently outperforms the other models on all benchmarks.

Specifically, MiniCPM achieves the highest Spearman correlations across all datasets, including notable results on STS12 (76.38%), STS13 (87.61%), and STS17 (89.96%). Gemma showed competitive performance but trailed slightly behind MiniCPM in every benchmark, with its highest correlation observed on STS17 (88.22%). Phi-2, on the other hand, exhibited the lowest performance among the three models, with its best result on STS17 (80.20%).

The overall performance metrics further illustrate this trend, with MiniCPM achieving an average Spearman correlation of $83.84\% \pm 4.27$, which is superior to Gemma's $81.56\% \pm 4.83$ and Phi-2's $70.89\% \pm 6.91$. These results indicate that while all models benefited from the fine-tuning process, MiniCPM's architecture and training regimen enabled it to achieve superior alignment with human judgment across diverse datasets.

In conclusion, the evaluation underscores MiniCPM's robustness and effectiveness in capturing semantic similarities, making it a reliable choice for tasks requiring nuanced understanding of textual data.

The fine-tuned versions of these three models have been published and are available for access here[6].

[1] https://alt.qcri.org/semeval2014/task10/
[2] https://alt.qcri.org/semeval2015/task2/
[3] https://alt.qcri.org/semeval2016/task1/
[4] https://alt.qcri.org/semeval2017/task1/
[5] https://tabilab.cmpe.boun.edu.tr/BIOSSES/DataSet.html

[6] https://huggingface.co/collections/trapoom555/small-lms-text-embedding-663b3ec87527788a577f6852

Fine-tuning Details	Value
Loss	InfoNCE
Batch Size	60
Learning Rate	5e-05
LoRA Rank	8
LoRA Alpha	32
LoRA Dropout	0.1
Mixed Precision	bf16
Max Epoch	1
Learning Rate Scheduler	CosineAnnealingLR
Learning Rate Warmup Steps	100
GPU	RTX3090
Num GPUs	4

Table 1: Contrastive fine-tuning configuration details.

Benchmark	Gemma	Phi-2	MiniCPM
STS12	75.80	61.62	**76.38**
STS13	85.45	71.87	**87.61**
STS14	80.08	60.46	**81.55**
STS15	85.02	71.18	**87.33**
STS16	83.33	74.77	**85.25**
STS17	88.22	80.20	**89.96**
STSBenchmark	85.61	79.46	**86.51**
BIOSSES	73.83	64.06	**80.05**
SICK-R	76.69	74.37	**79.87**
Overall	81.56 ± 4.83	70.89 ± 6.91	**83.84±4.27**

Table 2: Spearman correlations of cosine similarities of various models after SFT across different datasets.

4.2 Ablation Studies

4.2.1 Model performance before SFT

In this study, we aim to compare the performance of Gemma, Phi-2 and MiniCPM before and after applying SFT (fine-tuning). The results are detailed in Tables 2 and 3.

Table 3 presents the spearman correlations of cosine similarities for the models prior to SFT across various benchmarks. Gemma consistently outperforms Phi-2 and MiniCPM on all benchmarks, achieving the highest overall correlation of $56.63\% \pm 7.89$. Phi-2 and MiniCPM follow with overall correlations of $32.21\% \pm 11.40$ and $27.51\% \pm 12.76$, respectively.

Comparing the results before and after SFT, we observe significant performance improvements on all three models. Notably, MiniCPM demonstrates the most substantial improvement, increasing its overall correlation by approximately 56 points. This indicates that SFT has a pronounced positive impact on MiniCPM's performance.

In summary, while all models benefit from SFT, MiniCPM stands out as the top performer after SFT.

4.2.2 Impact of learning rate

In this study, we aim to study the impact of learning rate during training. Results from Table 4 shows that learning rate 5e-5 achieves the best result among the others, whereas learning rate 5e-3 achieves the lowest spearman correlations score, indicating the model suffers from training instability and is underfitting.

4.2.3 How does prompting affect model performance?

In our main experiments shown in Tables 2 and 3, we tested MiniCPM's embeddings without any additional prompts. Yet, it is known that employing prompting techniques may further leverage a lan-

Benchmark	Gemma	Phi-2	MiniCPM
STS12	**43.83**	23.04	7.27
STS13	**66.36**	20.79	18.38
STS14	**49.57**	17.06	15.04
STS15	**57.40**	24.56	32.24
STS16	**70.13**	48.68	39.79
STS17	**58.34**	41.43	33.63
STSBenchmark	**57.36**	37.87	33.91
BIOSSES	**48.67**	28.04	18.03
SICK-R	**58.02**	48.40	49.30
Overall	**56.63±7.89**	32.21 ± 11.40	27.51 ± 12.76

Table 3: Spearman correlations of cosine similarities of various models before SFT across different datasets.

Learning Rate	MiniCPM
5e-3	36.19 ± 8.91
5e-4	82.41 ± 5.01
5e-5	82.31 ± 5.27

Table 4: Experimental results of varying learning rate during training. The values shown are the average spearman correlations of cosine similarities across all 9 test datasets

Prompt	MiniCPM (tuned)	MiniCPM
No Prompt	<u>83.84</u>	27.51
Prompt 1	81.81	**34.76**
Prompt 2	83.03	26.92
Prompt 3	**84.32**	<u>28.79</u>

Table 5: Results of our prompt-tuning experiments. The values shown are the average spearman correlations of cosine similarities across all 9 test datasets under the specified prompt.

guage model's performance. Hence, we conducted an ablation study on MiniCPM with prompt-tuning.

We adopted the Explicit One-Word Limitation prompting technique (Jiang et al., 2023b), along with two other prompts:

1. "This sentence: {original_sentence} means in one word: "
2. "This sentence {original_sentence} means: "
3. "{original_sentence} is: "

During testing, {original_sentence} is replaced by the sentence from the current test case.

We tested all 3 prompts on both our fine-tuned and original MiniCPM models. The results are shown in Table 5.

As the results show, the original MiniCPM model performs best with Prompt 1, reaching a +7.25% gain in average performance. The original model experiences performance enhancements with both Prompt 1 and Prompt 3. In contrast, our fine-tuned MiniCPM model only achieves a minor +0.48% performance gain with Prompt 3, and is negatively impacted by both Prompt 1 and Prompt 2. Such results suggest that prompt-tuning is effective in certain conditions but harmful in others, and it is more effective to the original MiniCPM model than to our fine-tuned MiniCPM model. We hypothesize that this may be due to our fine-tuned model's preference towards the distribution of the original sentences instead of sentences augmented with prompts, since the model has been exposed to more sentences of the original format during fine-tuning.

4.2.4 Training data efficiency

In this study, we aim to study training data efficiency by measuring how much training data is required to achieve convergences. To do so, we measure evaluate the every 20 checkpoints until the Spearman correlation coverged since lesser training step equivalent to lesser training data used. The results are recorded in Figure 1.

Our results show that after 200 training steps, MiniCPM has already earned a average performance gain of +50% over our 9 test sets, indicating that it underwent the majority of its capability growth during the first 200 steps. Further-

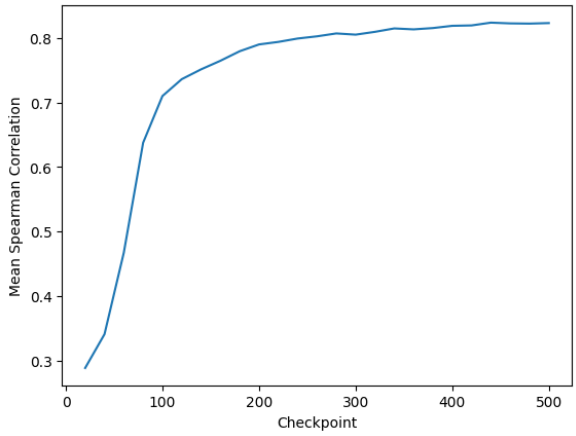

Figure 1: Average performance of model checkpoints across all 9 test sets. The values are the average spearman correlations of cosine similarities across all 9 test sets. The model converged after checkpoint 200.

more, the model begins to converge at around 200 steps, reaching an average performance of 82.31% at Checkpoint-500. This shows that MiniCPM demonstrates a relatively high training efficiency with great convergence speed.

4.2.5 Impact of hard negatives penalty

In the context of contrastive learning, the selection of hard negatives plays a critical role in enhancing the effectiveness of the training process. To study the impact of the hard negatives on the performance of MiniCPM after SFT, we remove the penalization on the hard negative samples by modifying the objective function in Equation 1 to become:

$$\min -\log \frac{e^{\text{sim}(\mathbf{h}_i, \mathbf{h}_i^+)/\tau}}{\sum_{j=1}^{N} \left(e^{\text{sim}(\mathbf{h}_i, \mathbf{h}_j^+)/\tau} \right)} \tag{2}$$

Table 6 compares the Spearman correlations of cosine similarities of the MiniCPM model with and without penalization on the hard negative samples. The results indicate that incorporating penalization generally improves performance across most benchmarks. Specifically, the MiniCPM model shows an increase in performance on all benchmarks except STS17 when hard negative samples are penalized.

For instance, the Spearman correlation for STS13 improves from 85.98% to 87.61%, which is a notable increase. Similarly, STS15 sees a rise from 85.66% to 87.33%, and STS14 improves from 78.52% to 81.55%. These results suggest that penalizing hard negative samples helps the model better distinguish between similar and dissimilar pairs, leading to higher correlation scores.

However, there is a slight decrease in performance for STS17, where the correlation drops from 90.23% to 89.96%. This anomaly might indicate that for some datasets, the penalization might not always lead to improvements, potentially due to the nature of the data or the specific characteristics of the benchmark.

Overall, the average performance across all benchmarks increases from 81.97% to 83.84%. This demonstrates that penalizing hard negative samples not only boosts the average performance but also leads to more consistent results, as evidenced by the lower standard deviation.

Benchmark	**MiniCPM (No Hard Neg)**	**MiniCPM**
STS12	75.49	**76.38**
STS13	85.98	**87.61**
STS14	78.52	**81.55**
STS15	85.66	**87.33**
STS16	84.69	**85.25**
STS17	**90.23**	89.96
STSBenchmark	85.48	**86.51**
BIOSSES	78.31	**80.05**
SICK-R	73.35	**79.87**
Overall	81.97 ± 5.37	**83.84± 4.27**

Table 6: Spearman correlations of cosine similarities of MiniCPM without penalty on the hard negative samples. The 'MiniCPM (No Hard Neg)' column represents results from training without penalization on hard negatives, whereas the 'MiniCPM' column represents results from training with hard negative penalty.

5 Conclusion

In this project, we enhanced the text embedding capabilities of MiniCPM through contrastive fine-tuning using the NLI dataset. Our results demonstrate that MiniCPM achieved a significant performance gain of 56.33% and outperformed 2 other models, Gemma and Phi-2, across 9 STS datasets. Additionally, we conducted multiple ablation studies to delve deeper into our method, examining aspects such as prompt-tuning, training efficiency, and the effects of incorporating a hard negatives penalty in the objective function. Our research contributes to the enhancement of text embedding qualities in smaller-scale LLMs, making them more

robust and reliable for further applications.

6 Acknowledgement

We would like to formally acknowledge the Natural Language Processing Spring 2024 course for providing us with the foundational knowledge and the opportunity to undertake this project. We also extend our sincere gratitude to Professor Huaping Liu and the Tsinghua Knowledge Engineering Group for supplying the essential computational resources for our work.

References

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

Eneko Agirre, Carmen Banea, Claire Cardie, Daniel Cer, Mona Diab, Aitor Gonzalez-Agirre, Weiwei Guo, Rada Mihalcea, German Rigau, and Janyce Wiebe. 2014. Semeval-2014 task 10: Multilingual semantic textual similarity. In *Proceedings of the 8th international workshop on semantic evaluation (SemEval 2014)*, pages 81–91.

Eneko Agirre, Daniel Cer, Mona Diab, and Aitor Gonzalez-Agirre. 2012. Semeval-2012 task 6: A pilot on semantic textual similarity.* sem 2012: The first joint conference on lexical and computational semantics—. In *Proceedings of the Sixth International Workshop on Semantic Evaluation (SemEval 2012), Montréal, QC, Canada*, pages 7–8.

Eneko Agirre, Daniel Cer, Mona Diab, Aitor Gonzalez-Agirre, and Weiwei Guo. 2013. * sem 2013 shared task: Semantic textual similarity. In *Second joint conference on lexical and computational semantics (* SEM), volume 1: proceedings of the Main conference and the shared task: semantic textual similarity*, pages 32–43.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners.

Sumit Chopra, Raia Hadsell, and Yann LeCun. 2005. Learning a similarity metric discriminatively, with application to face verification. In *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05)*, volume 1, pages 539–546. IEEE.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding.

Raia Hadsell, Sumit Chopra, and Yann LeCun. 2006. Dimensionality reduction by learning an invariant mapping. In *2006 IEEE computer society conference on computer vision and pattern recognition (CVPR'06)*, volume 2, pages 1735–1742. IEEE.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for nlp.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models.

Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. 2024. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023a. Mistral 7b.

Ting Jiang, Shaohan Huang, Zhongzhi Luan, Deqing Wang, and Fuzhen Zhuang. 2023b. Scaling sentence embeddings with large language models.

Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. 2023. Textbooks are all you need ii: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*.

Niklas Muennighoff. 2022. Sgpt: Gpt sentence embeddings for semantic search. *arXiv preprint arXiv:2202.08904*.

Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2022. Mteb: Massive text embedding benchmark. *arXiv preprint arXiv:2210.07316*.

Jianmo Ni, Gustavo Hernandez Abrego, Noah Constant, Ji Ma, Keith B Hall, Daniel Cer, and Yinfei Yang. 2021. Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models. *arXiv preprint arXiv:2108.08877*.

Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*.

PyTorch. Cosineannealinglr. https://pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.CosineAnnealingLR.html. Accessed: 2024-05-05.

Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*.

Florian Schroff, Dmitry Kalenichenko, and James Philbin. 2015. Facenet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 815–823.

Kihyuk Sohn. 2016. Improved deep metric learning with multi-class n-pair loss objective. *Advances in neural information processing systems*, 29.

Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models.

Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*.

A Fine-tuning Details

In contrastive learning, a large batch size is required to improve training stability. In this work, we aim to enhance text embedding performance with limited computational resources. Therefore, we implement several training techniques to achieve this goal. We employ LoRA (Hu et al., 2021), a parameter-efficient fine-tuning method, to significantly decrease the GPU VRAM requirements for a single batch. Consequently, more batches can fit into a single GPU. To further reduce GPU VRAM demand, we adopt the bf16 mixed-precision training technique, which also increases training speed while maintaining numerical stability. Additionally, we leverage multi-GPU processing to amplify batch size across GPUs using the distributed data parallelism (DDP) technique. Furthermore, we incorporate a learning rate scheduler with CosineAnnealingLR (PyTorch) and learning rate warm-up to optimize the training process.

All fine-tuning details listed in Table 1. With these settings, the model can be effectively fine-tuned using limited resources, ensuring efficiency and cost-effectiveness. The entire fine-tuning process involves 13,780 optimization steps to cover the entire dataset.

Figure 2 illustrates the fine-tuning loss curves of the three selected models: Gemma, Phi-2, and MiniCPM. These curves show that the loss converges very quickly at the beginning of the fine-tuning process, indicating a rapid improvement in model accuracy during the initial stages. We observed this phenomenon and further examined the training data efficiency in the ablation study 4.2.4.

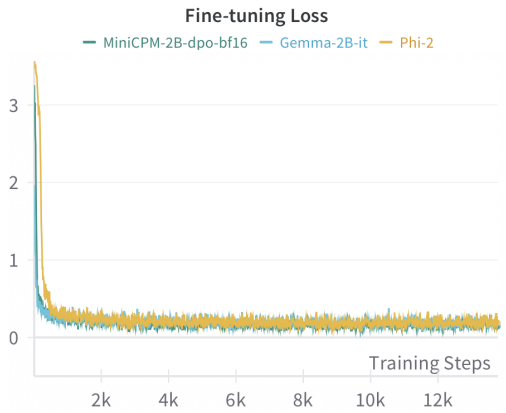

Figure 2: Fine-tuning loss over training steps.